GitHub: Version Control and Collaborative Development

Presented by: [Student Names] ▪ Department of Data Science

A practical introduction to Git and GitHub for BSc Data Science & Computer Science students (beginners)

Agenda

Foundations

What is Version Control ▪ Git vs GitHub ▪ Key terms

Practical Flow

Three zones ▪ Save cycle ▪ Create repo ▪ Connect local → GitHub

Collaboration

Branches ▪ Pull requests ▪ Merge conflicts ▪ Issues

Advanced

Open source workflow ▪ GitHub Actions ▪ Real-world uses

We'll mix diagrams, command snippets, and screenshots to build practical intuition.

What is Version Control?

Version control is a system that records changes to files over time so you can recall specific versions later. It prevents chaotic filenames (Project_Final_v2_FINAL.doc) and enables safe collaboration — multiple people can work concurrently without losing history.

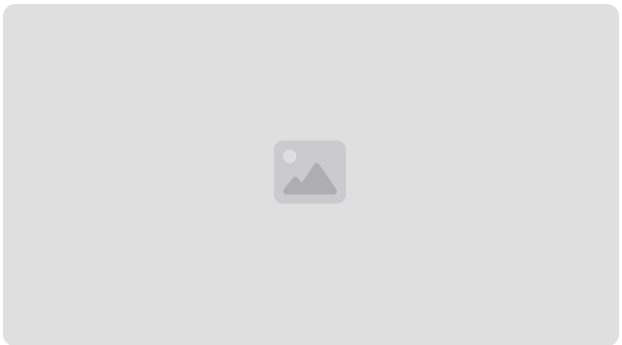
Why it matters

Track edits, recover prior states, and understand who changed what and when.

Collaboration

Merge contributions from teammates instead of juggling multiple files.

Benefits of Version Control



Track changes

Maintain full history and authorship.

Collaboration

Work concurrently and merge contributions cleanly.

Backup & recovery

Revert accidental changes and restore prior states.

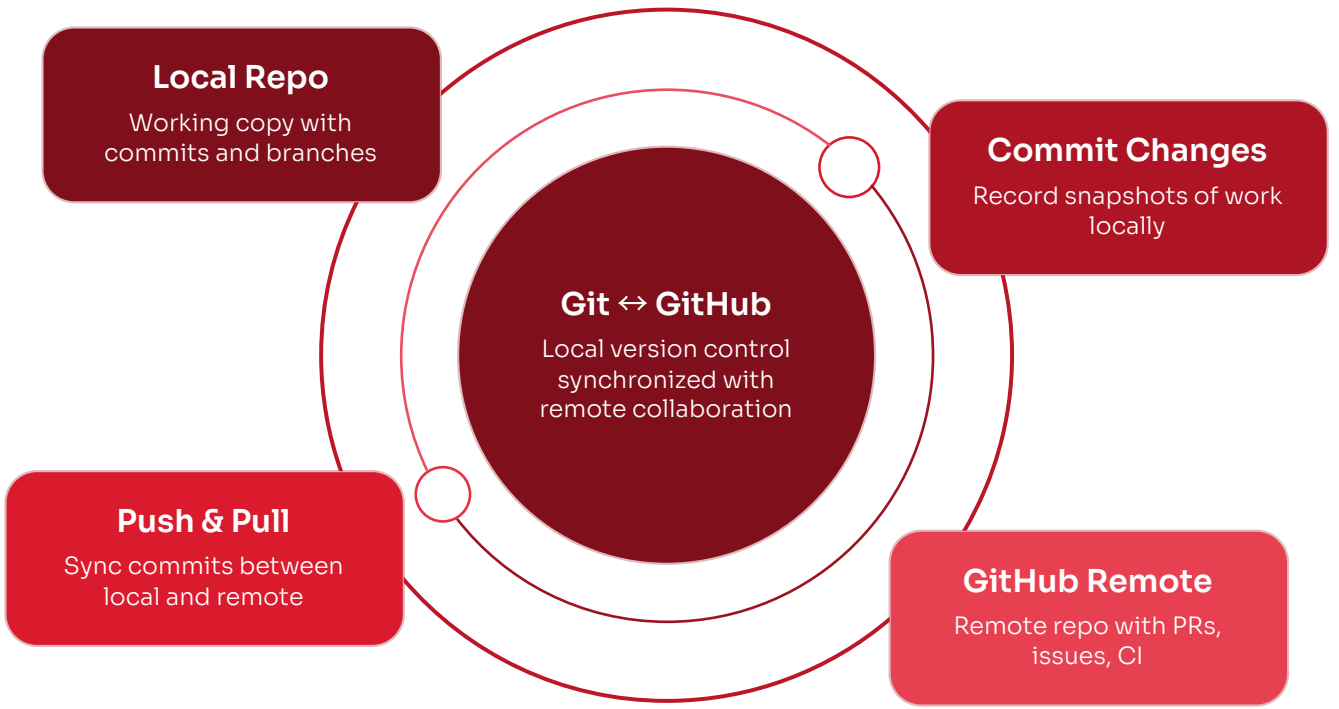
Rollback

Return to a stable version quickly when needed.

Pro tip: Good commit messages make history useful—write descriptive, concise messages.

Git vs GitHub

Git	GitHub
Distributed version control system	Cloud platform for hosting repositories
Works locally (on your machine)	Supports collaboration, PRs, issues, CI/CD
Records commits, branches, merges	Provides remote storage, web UI, and integrations



Think of Git as the engine and GitHub as the collaborative dashboard and remote backup.

Repository

Project folder tracked by Git (local or remote).

Commit

Snapshot of changes with a message and author.

Clone

Copy of a remote repo locally.

Fork

Copy on GitHub for contributions.

Push

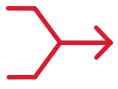
Uploads local commits to remote.

Pull

Fetches and merges remote changes.

Branch

Isolates work in a separate line of development.



Merge

Integrates changes into another branch.

Key Git Terminologies

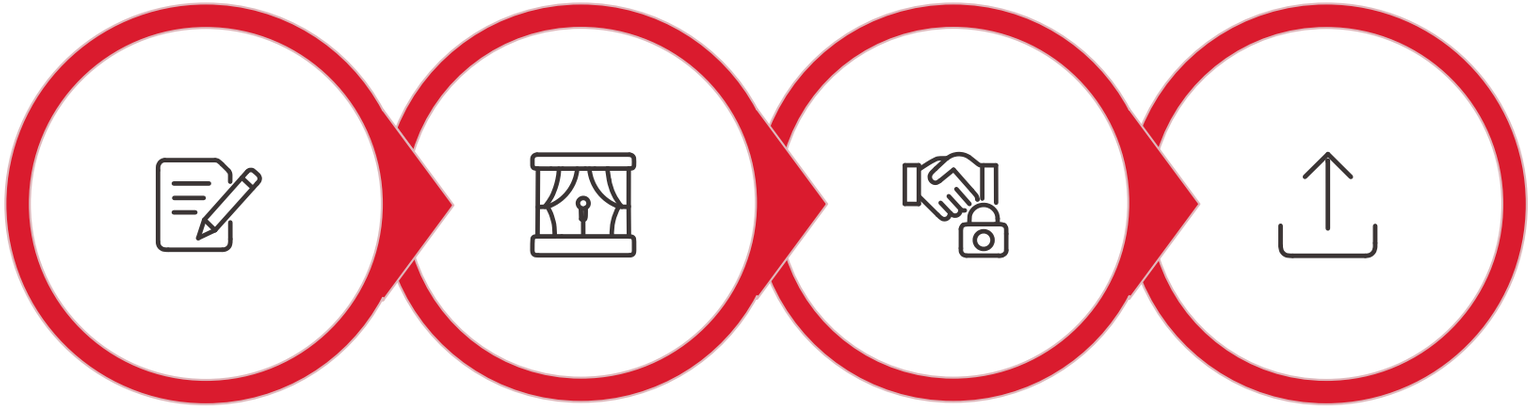
Use the icons as quick visual cues for the most common Git actions and concepts.

Three Zones of Git

Working Directory: where you edit files. Staging Area: files prepared for commit (git add). Local Repository: committed snapshots (git commit) stored in .git.

Visualise the staging area as a shopping cart: you add chosen items before checkout (commit).

Git Save Cycle (Common Commands)



Edit File **git add** **git commit** **git push**

01

Edit

Modify code or documents in your working directory.

02

Stage (git add)

Select changes to include in the next commit.

03

Commit (git commit)

Create a local snapshot with a descriptive message.

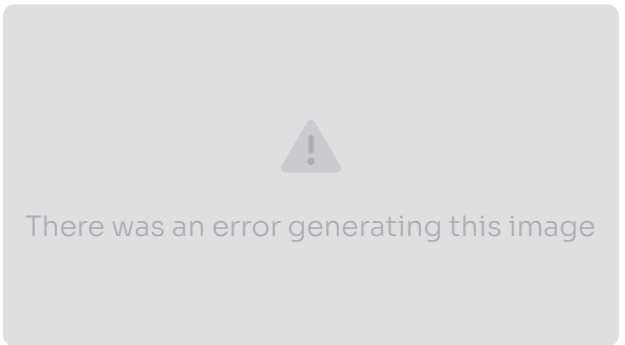
04

Share (git push)

Send commits to GitHub so teammates can access them.

Commands example: git init ▪ git add . ▪ git commit -m "First commit" ▪ git remote add origin URL ▪ git push -u origin main

From Zero to GitHub: Create & Connect

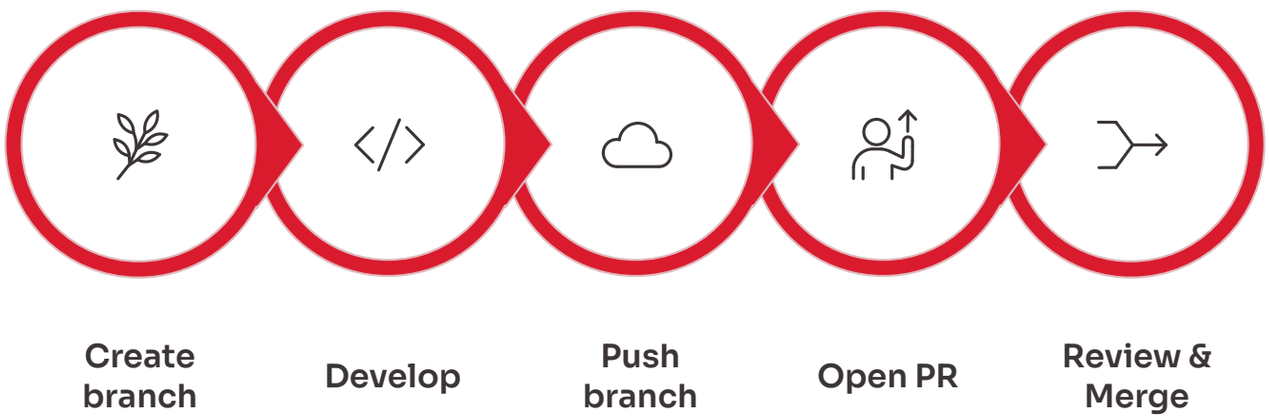


Steps (concise)

1. Create GitHub account and click New → Repository
2. Name repo, choose Public or Private, add README
3. On local machine: git init → git add . → git commit
4. Link remote: git remote add origin <URL>
5. Push: git push -u origin main

Public vs Private: Public is visible to others (good for portfolios). Private keeps code restricted.

Branching, PRs & Collaboration (Compact Workflow)



Branching Basics

Use branches for independent features—safe, isolated workspaces. Commands: git branch <name>, git switch <name>.

Pull Requests

PR = request to merge. Enables code review, discussion, and CI checks before merging into main.

Merge Conflicts

Occur when same lines change in parallel. Resolve manually, test, commit resolved files.

Quick commands: git checkout -b feature-name ▪ git push -u origin feature-name ▪ Open PR on GitHub